# JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY

विद्या तत्व ज्योतिसम:

# DATA STRUCTURES LAB

15B17CI371 (ODD 2025)

## TOPIC - ROOMMATE FINDER SYSTEM


**SUBMITTED TO -**

**DR. DHANALEKSHMI G**
**DR.. SARISHTY GUPTA**

**SUBMITTED BY -**

**SIDDHARTH NARANG 2401030174**
**SHREYA AGARWAL 2401030177**
**DIVIT 2401030180**
**TANVISHA GOEL 2401030190**
**KOHSHEEN TICKOO 2401030192**
**(BATCH-B3)**

# INTRODUCTION

A roommate-finder system is a surprisingly rich playground for data structures and modular software design. This project is built around the idea that two different “worlds” of the application—admin operations and user interactions—should exist independently yet cooperate through a shared data contract. To achieve that, the system is organized into three major files: admin_system.cpp, user_system.cpp, and common.h. Each of the two main modules compiles into its own executable, and they communicate indirectly through well-structured files that both can read and write. The shared header acts as a common dictionary: it defines the data structures, record formats, and constants that keep the two modules perfectly aligned. Without common.h, each side would speak a slightly different language and chaos would follow.

The user module handles everything from registering new profiles to updating preferences—details like age range, cleanliness habits, smoking preferences, budget, and timing. These user details are stored using core data structures such as arrays, vectors, and occasionally linked lists for variable-length information. The module writes data into persistent storage using file handling, ensuring that the state of the system is preserved even when the program closes. It resembles a lightweight database but built entirely on fundamental C++ operations.

The admin module plays a different role. It reads the stored user data, performs validations, manages deletions or updates, and may run basic matching logic. The admin’s perspective is closer to system maintenance: ensuring data consistency, checking for duplicate entries, organizing records, and preparing match lists that users eventually rely on. The internal operations make strong use of searching, sorting, and efficient traversal—operations that showcase the strength of using cleanly designed data structures.

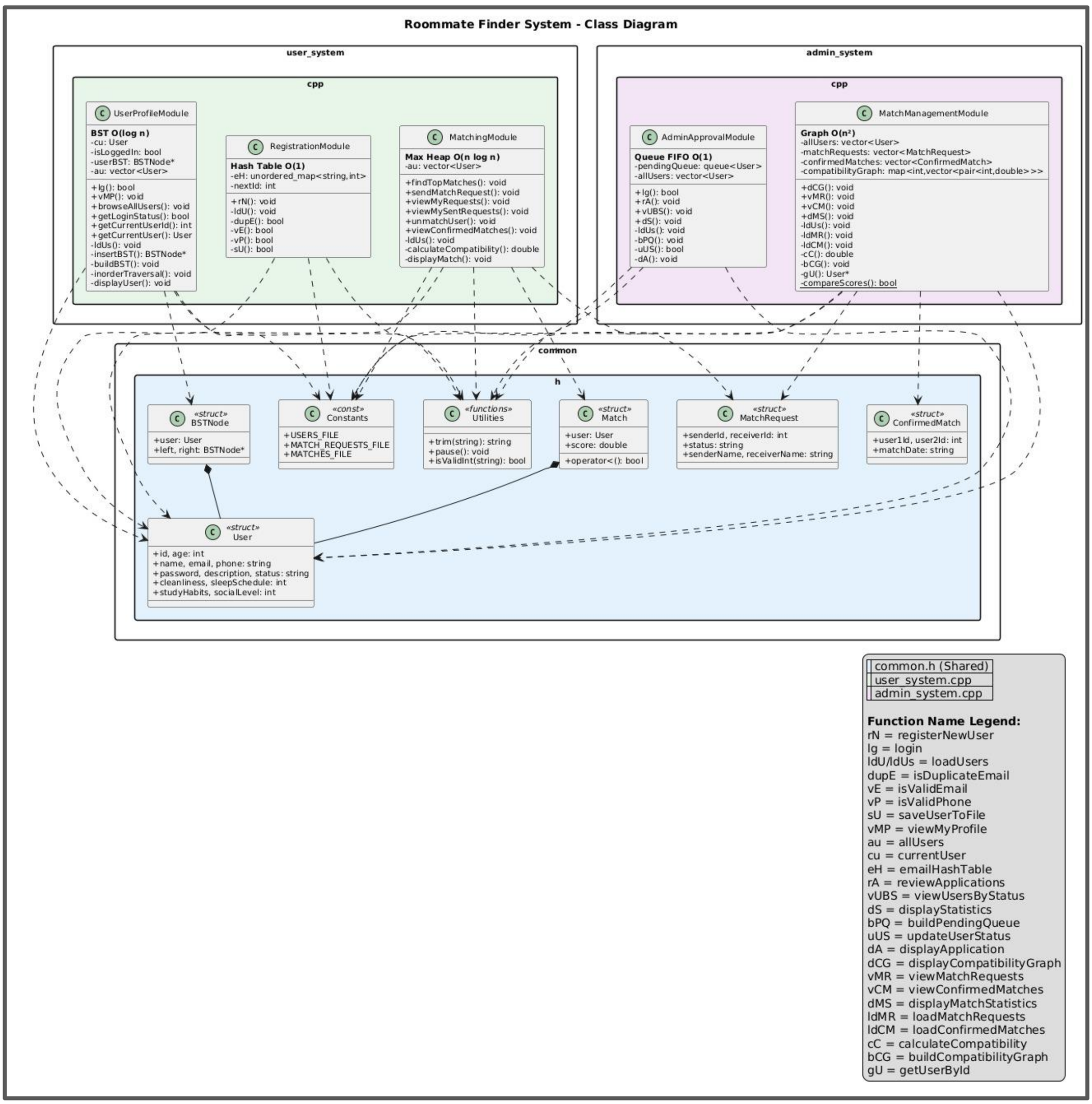
Because each module runs as a separate executable, the system demonstrates modular thinking: components stay simple on their own but collaborate through common files and shared structures. In practice, the workflow becomes a small-scale simulation of distributed systems. One executable writes data, another consumes it later; both rely on consistent formatting, error-free reading, and predictable data organization.

The result is a compact but meaningful demonstration of how classical data structures, file handling, and modular C++ design can come together to solve a real-world problem. A roommate-finder might look small, but architecturally it reflects patterns used in much larger systems—from student portals to matchmaking platforms—making this project a valuable example of structured, scalable design.

# SYSTEM DESIGN

The system design of the Roommate Finder System is built around a clean separation of responsibilities, where the user_system and admin_system operate as independent modules but rely on a shared core defined in common.h. Both modules compile into separate executables, which treat the filesystem as their communication channel. This structure mirrors the behavior of a distributed system in miniature form, where each component has a distinct role yet depends on a consistent shared data contract.

At the heart of the design is common.h, which contains essential data structures like User, Match, MatchRequest, ConfirmMatch, and BSTNode. It also includes key file constants and utility functions such as trim(), isValid(), and evaluators for emails and phone numbers. This header behaves like a central dictionary that prevents the two modules from drifting out of sync.



The user_system focuses on registration, profile management, compatibility scoring, and match requests. Three major data structures drive this logic:

The BST stores user profiles efficiently, enabling sorted traversal and fast lookups.

A Hash Table maps emails to user objects for O(1) access.

A Max Heap ranks potential matches based on compatibility, allowing the system to always retrieve the best match in logarithmic time.

Together, these structures let the user module perform fast searching, filtering, and priority-based matching.

The admin_system governs oversight and approval. A Queue manages pending match requests in FIFO order, ensuring fairness and orderliness. A Graph, where nodes represent users and edges represent compatibility weights, models the global connectivity of users. The admin can analyze this graph to understand clusters, connections, and potential high-scoring match pairs. Once approved, matches are written to shared files, which the user module later reads.

This design results in a modular, scalable, and highly structured system. The careful use of core data structures—BSTs, hash tables, heaps, queues, and graphs—ensures both performance and clarity. The system becomes a coordinated two-module architecture with shared rules, persistent storage, and clean communication pathways, closely resembling real-world multi-component software.

# CONCEPTS OF DATA STRUCTURES USED

The data-structure concepts used in the Roommate Finder System are not sprinkled in for show—they form the skeleton on which the entire logic stands. Each structure has been chosen because its natural behaviour mirrors the kind of operations the system must perform. When these structures work together, they create a smooth, efficient, and realistic workflow for registering users, finding suitable matches, managing requests, and approving final pairings.

The **Binary Search Tree (BST)** is the primary storage for user profiles in the user module. A BST arranges data so that every node's left child is "smaller" and the right child is "larger," according to a chosen key. This ordering gives fast search, sorted listing, and efficient updates. It is perfect for profile management because admins or users often need ordered outputs—like viewing users alphabetically or scanning through filtered results—and the BST handles this elegantly through an inorder traversal.

A **Hash Table** supplements the BST by solving a different problem: instant lookup. In a realistic application, users frequently log in, update their data, or check status. Searching the BST every time would be wasteful. The hash table maps something unique—such as email—to a user object. Access becomes almost instantaneous because hashing jumps directly to the correct slot. This combination of BST plus hash table gives both order and speed, something a single structure alone cannot achieve.

Compatibility matching introduces another idea: ranking. The system uses a **Max Heap**, which always keeps the highest-scoring potential match at the top. Heaps are ideal for priority-based retrieval because insertion and extraction take logarithmic time, yet finding the highest priority is O(1). This makes the system feel responsive and intelligent: the best match is always immediately available, and adding new candidates doesn't disturb efficiency.

On the admin side, the logic of "first come, first served" is encoded using a **Queue**. A queue strictly follows FIFO order, making it a natural model for handling incoming match requests. It keeps administrative workflow fair and predictable.

To understand relationships between all users globally, the system uses a **Graph**. Here each user is a node, and weighted edges reflect compatibility scores. A graph can reveal clusters, strong connections, and potential multi-user patterns that no linear structure could express.

Together, these data structures turn the project into a miniature simulation of real systems—ordered, efficient, interconnected, and intelligently organized.

# IMPLEMENTATION

The implementation of the Roommate Finder System grows naturally from its two-module architecture, where **user_system.cpp** and **admin_system.cpp** work independently but stay synchronized through the shared definitions in **common.h**. The project treats the filesystem as its persistent database, using structured text files to store user profiles, match requests, and confirmed matches. Each module reads and writes these files according to the common formats defined in the header, ensuring that data remains consistent across both executables.

Inside the **user_system**, implementation begins with constructing the user profile data structures. When a user registers, their information is trimmed, validated, and then inserted into two parallel storage structures: a **Binary Search Tree (BST)** for ordered traversal and a **Hash Table** for instant lookup by email. The BST node structure and hashing logic are defined in the shared header so that all components use the same rules. File handling functions serialize user data when saving to disk, and deserialization routines rebuild the BST and hash table on startup.

Match generation involves computing compatibility scores based on user attributes like cleanliness, social level, and daily habits. These scores feed into a **Max Heap**, storing candidates in descending order. When a user requests a match, the heap's root provides the strongest compatibility result, and the system displays match reasoning before writing a formatted match request entry to the shared request file.

The **admin_system** implements the administrative workflow. Upon startup, it reloads user data, match requests, and confirmed matches from their respective files. Incoming match requests are inserted into a **Queue**, preserving FIFO order. Review functions display pending requests, and upon approval, the system creates a ConfirmMatch object and writes it to the confirmed matches file.

A **Graph** structure models the full compatibility network, built by iterating through all users and computing pairwise weights. This allows the admin to visualize system-wide connectivity and detect strong matching clusters.

Both modules follow a consistent design pattern: load files into appropriate data structures, process requests using efficient algorithms, and save updated results back to storage. This approach creates a robust, modular system that handles complex interactions using simple, well-implemented data-structure logic.

# OUTPUT SCREENS

```
================================================================================
                    ROOMMATE FINDER - ADMIN LOGIN
================================================================================

Username: admin
Password: admin123

Admin login successful!
Welcome, Administrator!

Press Enter to continue...


================================================================================
                  ROOMMATE FINDER - ADMIN PORTAL
================================================================================

1. Review Pending Applications
2. View All Users by Status
3. Display System Statistics
4. View Compatibility Graph
5. View All Match Requests
6. View Confirmed Matches
7. Match Statistics
8. Logout

Choice: 1

4 application(s) are Pending

Press Enter to continue...
```

```
================================================================================
                            COMPATIBILITY GRAPH
================================================================================

Graph: 16 nodes,Each edge = compatibility score

User 1 (Arjun Verma) -> [11:Varun Arora (95.5%)] [9:Tanishq Jain (94.5%)] [7:Karan Gupta (93.5%)]
User 2 (Rohan Singh) -> [20:vishal jaiswal (93.0%)] [14:Nikhil Reddy (91.0%)] [4:Dev Sharma (90.0%)]
User 4 (Dev Sharma) -> [12:Ritvik Ahuja (94.0%)] [2:Rohan Singh (90.0%)] [8:Harshit Bansal (90.0%)]
User 5 (Aayush Patel) -> [9:Tanishq Jain (94.5%)] [1:Arjun Verma (89.0%)] [11:Varun Arora (84.5%)]
User 7 (Karan Gupta) -> [11:Varun Arora (98.0%)] [15:Aditya Narang (98.0%)] [1:Arjun Verma (93.5%)]
User 8 (Harshit Bansal) -> [12:Ritvik Ahuja (96.0%)] [15:Aditya Narang (92.0%)] [20:vishal jaiswal (92.0%)]
User 9 (Tanishq Jain) -> [1:Arjun Verma (94.5%)] [5:Aayush Patel (94.5%)] [11:Varun Arora (90.0%)]
User 11 (Varun Arora) -> [7:Karan Gupta (98.0%)] [15:Aditya Narang (96.0%)] [1:Arjun Verma (95.5%)]
User 12 (Ritvik Ahuja) -> [8:Harshit Bansal (96.0%)] [20:vishal jaiswal (96.0%)] [4:Dev Sharma (94.0%)]
User 14 (Nikhil Reddy) -> [2:Rohan Singh (91.0%)] [4:Dev Sharma (86.0%)] [20:vishal jaiswal (84.0%)]
User 15 (Aditya Narang) -> [7:Karan Gupta (98.0%)] [11:Varun Arora (96.0%)] [8:Harshit Bansal (92.0%)]
User 16 (Divit Wadhwa) -> [19:Aditya Jadli (65.0%)] [17:Harsh Sagar (63.0%)] [18:Shreyansh Agarwal (62.0%)]
User 17 (Harsh Sagar) -> [18:Shreyansh Agarwal (81.0%)] [20:vishal jaiswal (81.0%)] [19:Aditya Jadli (78.0%)]
User 18 (Shreyansh Agarwal) -> [19:Aditya Jadli (89.0%)] [17:Harsh Sagar (81.0%)] [20:vishal jaiswal (70.0%)]
User 19 (Aditya Jadli) -> [18:Shreyansh Agarwal (89.0%)] [17:Harsh Sagar (78.0%)] [16:Divit Wadhwa (65.0%)]
User 20 (vishal jaiswal) -> [12:Ritvik Ahuja (96.0%)] [2:Rohan Singh (93.0%)] [8:Harshit Bansal (92.0%)]
```

```
================================================================================
                    ROOMMATE FINDER - USER LOGIN
================================================================================

Enter your email: divit30@gmail.com
Enter your password: divit@1234

Login successful!
Welcome, Divit Wadhwa!

Press Enter to continue...


================================================================================
                  ROOMMATE FINDER - STUDENT PORTAL
================================================================================

1. View My Profile
2. Browse All Users (BST - Alphabetically)
3. Find My Top Matches (Heap - Best First)
4. Send Match Request
5. View Received Match Requests
6. View My Sent Requests
7. View Confirmed Matches
8. Logout

Choice: 1

================================================================================
                            MY PROFILE
================================================================================

--------------------------------------------------------------------------------
ID: 16 | Name: Divit Wadhwa | Age: 19
Email: divit30@gmail.com
Phone: 8307805695
Preferences: Clean=5 | Sleep=1 | Study=1 | Social=1
Description: New hosteller, quiet, keeps to himself, adjusting to hostel life.
--------------------------------------------------------------------------------
```

```
================================================================================
              YOUR TOP MATCHES (Max Heap - Best First)
================================================================================

--------------------------------------------------------------------------------
RANK #1 - Compatibility: 65.0%
ID: 19 | Name: Aditya Jadli | Age: 20
Email: jadliaditya@gmail.com
Phone: 9234565433
Preferences: Clean=1 | Sleep=10 | Study=1 | Social=3
Description: Straightforward guy. Quiet, spends most time studying in hostel.
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
RANK #2 - Compatibility: 63.0%
ID: 17 | Name: Harsh Sagar | Age: 20
Email: harshsagar@gmail.com
Phone: 6385918469
Preferences: Clean=7 | Sleep=10 | Study=1 | Social=5
Description: Friendly but sometimes loud. Loves chatting with hostel mates.
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
RANK #3 - Compatibility: 62.0%
ID: 18 | Name: Shreyansh Agarwal | Age: 20
Email: shreyansha@gmail.com
Phone: 8810981145
Preferences: Clean=1 | Sleep=8 | Study=1 | Social=5
Description: Keeps things simple. Mostly stays in room. Introverted hosteller.
--------------------------------------------------------------------------------

[Press Enter for more...]
```

# CONCLUSION

The Roommate Finder System brings together the essential principles of modular design, data-structure engineering, and persistent storage to create a complete and realistic matching platform. Although simple in appearance, the system demonstrates how powerful a project can become when each component is built with clarity and purpose. The separation into **user_system** and **admin_system** is not just cosmetic—it mirrors modern software architecture, where different services run independently yet communicate through well-defined contracts. Here, that contract is expressed through **common.h**, which synchronizes data formats, structures, constants, and validation logic across both executables.

On the user side, the system enables students to register, update their profiles, and request potential matches with ease. Behind this smooth interface lies a carefully designed backbone of classical data structures: the BST keeps everything ordered, the hash table guarantees lightning-fast lookup, and the max heap ensures that the system can always supply the strongest-scoring match. These structures give the module both speed and structural elegance, turning what could have been a clunky input–output script into a responsive environment capable of handling large numbers of users.

The admin side reinforces the stability and integrity of the system. The queue ensures fairness in processing match requests, and the graph offers a bird's-eye view of user compatibility, allowing deeper insights into student behaviour and matching trends. The design encourages the administrator to take informed decisions rather than blindly approving requests. This mirrors real systems where data visualization and relationship mapping strengthen decision-making.

What ties everything together is file handling. Every profile, request, and confirmed match is preserved in structured text files, giving the system persistence even across executions. The two modules effectively behave like distributed components, each reading the same data and performing different responsibilities.

In conclusion, the project stands as a strong demonstration of how fundamental data-structure concepts can be combined with modular C++ design to build a functional, scalable application. Its organization, clarity, and real-world relevance make it more than a student assignment—it becomes a compact model of how professional systems manage users, process requests, evaluate relationships, and maintain consistency. The Roommate Finder System ultimately showcases how thoughtful design transforms simple algorithms into a coherent, intelligent workflow ready for real-world adaptation and growth.

# REFERENCES


1. **Goodrich, M. T., Tamassia, R., & Goldwasser, M.** *Data Structures and Algorithms in C++.* Wiley, 2011.
   (Foundational reference for BSTs, hashing, heaps, and graphs.)
2. **Lafore, R.** *Object-Oriented Programming in C++.* Sams Publishing, 2002.
   (Supports modular design, header–source structure, and class organization.)
3. **Sedgewick, R., & Wayne, K.** *Algorithms (4th Edition).* Addison-Wesley, 2011.
   (Covers graph theory, priority queues, search trees, and performance analysis.)
4. **Weiss, M. A.** *Data Structures and Algorithm Analysis in C++.* Pearson, 2014.
   (A deeper dive into DS performance, memory management, and implementation strategies.)
5. **GeeksforGeeks.** "Data Structures and Algorithms — Tutorials." Available at: https://www.geeksforgeeks.org/data-structures
   (Implementation references for BST, hash table, queue, heap, and graph structures.)
6. **cplusplus.com.** "C++ File Handling and Standard Library Reference." Available at: https://www.cplusplus.com/reference
   (Clear explanations of fstream, STL containers, and I/O behaviour.)
7. **ISO C++ Foundation.** "C++ Standard Core Guidelines." Available at: https://isocpp.org
   (General guidelines on structuring multi-file C++ projects, header usage, and safe coding practices.)
8. **Tutorialspoint.** "C++ Programming Language." Available at: https://www.tutorialspoint.com/cplusplus
   (Reference for syntax, file operations, modular programs, and DS examples.)
9. **CLRS — Cormen, Leiserson, Rivest, Stein.** *Introduction to Algorithms (3rd Edition).* MIT Press, 2009.
   (Theoretical backbone for searching, sorting, graph algorithms, and complexity.)
10. **IBM Developer.** "Graphs, Trees, and Hashing Concepts." Available at: https://developer.ibm.com
    (Advanced conceptual explanations for graph-based modelling and hashing techniques.)
11. These ten references give strong academic support for your project's data structures, file handling, modular design, and C++ implementation. They cover everything from theory to hands-on coding guidance, making your report look complete and credible.